

The Sketching Approach to Program Synthesis

Armando Solar-Lezama

Massachusetts Institute of Technology

Abstract. Sketching is a new form of localized software synthesis that aims to bridge the gap between a programmer’s high-level insights about a problem and the computer’s ability to manage low-level details. In sketching, the programmer uses partial programs to describe the desired implementation *strategy*, and leaves the low-level details of the implementation to an automated synthesis procedure. This paper describes the sketching approach to program synthesis, including the details of the SKETCH language and synthesizer. The paper will then describe some of the techniques that make synthesis from sketches possible, and will close with a brief discussion of open problems in programmer guided synthesis.

1 Introduction

Sketching is a new form of localized program synthesis that allows programmers to express their high-level insights about a problem by writing a *sketch*—a partial program that encodes the structure of a solution while leaving its low-level details unspecified. In addition to the sketch, programmers provide the synthesizer with either a reference implementation or a set of test routines that the synthesized code must pass. The SKETCH synthesis engine is able to derive the missing details in the sketch to produce a working implementation that satisfies the correctness criteria established by the programmer. By combining the programmers insight expressed in the sketch with the synthesizer’s ability to reason exhaustively about all the low-level details of the problem, sketching allows complex algorithms to be implemented with minimal human effort.

To illustrate the use of sketching on an interesting programming problem, consider the problem of reversing a linked list. It is relatively easy to write a recursive solution to this problem, but the performance of the simple implementation is likely to be unacceptable. A more efficient implementation must use a loop instead of recursion, and must construct the new list backwards to avoid the linear storage. Sketching allows the programmer to express these insights as a partial program without having to think too much about the details of the implementation.

The sketch for this problem is shown in Figure 1. The body of `reverseEfficient` encodes the basic structure of the solution: allocate a new list, and perform a series of conditional pointer assignments inside a while loop. In order to define the space of possible conditionals and assignments, the sketch uses regular expression notation to define sets of expressions in lines 1 to 3. The sketch, in short, encodes

```

1: #define LHS { | tmp | ( l | nl ).( h | t ).( next )? | }
2: #define LOC { | LHS | null | }
3: #define COMP { | LOC ( == | != ) LOC | }

list reverseEfficient(list l){
4:   list nl = new list();
5:   node tmp = null;
6:   bit c = COMP;
7:   while(c){
8:     repeat(??)
9:       if( COMP ){ LHS = LOC; }
10:      c = COMP;
   }
}

// test harness
void main(int n){
  if(n >= N){ n = N-1; }
  node[N] nodes = null;
  list l = newList();
  //Populate the list, and
  //write its elements
  //to the nodes array
  populateList(n, l, nodes);

  l = reverseSK(l);

  //Check that node i in
  //the reversed list is
  //equal to nodes[n-i-1]
  check(n, l, nodes);
}

```

Fig. 1. Complete sketch and specification for the linked list reversal problem

everything that can be easily said about the implementation, and constrains the search space enough to make synthesis tractable.

Together with the sketch, the programmer must provide a specification that describes the correct behavior of the reversal routine. The SKETCH synthesizer allows the user to provide specifications in the form of parameterized or non-deterministic test harnesses. Figure 1 shows the test harness for the list reversal; the synthesizer will guarantee that the harness succeeds for all values of $n \leq N$. On a laptop, the complete synthesis process takes less than a minute for $N=4$.

The sketching approach to synthesis applies just as effectively to concurrent programs, where the synthesizer’s ability to orchestrate low-level details is even more desirable given the difficulty that programmers have in reasoning about the effects of different thread interleavings. To illustrate the use of sketching in this domain, consider the problem of implementing a barrier. A barrier is a synchronization mechanism that forces a group of threads to stop at a particular point in the program until all threads in the group have reached this point. Figure 2 shows the specification for a barrier. The specification is given as a simple test method where N threads repeatedly call the next method of the barrier. The test uses a two dimensional array to check that no thread races past the barrier before its left and right neighbors have reached the next method.

```

struct barrier{
  int nthreads; bit sense; bit[N] localSenses;
}
void main(){
  bit[N][T] grid = 0;
  barrier b = newBarrier(N);
  fork(int thread; N){
    for(int i=0; i<T; ++i){
      grid[i][thread] = 1;
      next(b, thread);
      assert grid[i][left(thread)] && grid[i][right(thread)];
    }
  }
}

```

Fig. 2. Specification for a sense reversing barrier